

HIGH-LEVEL DESIGN (HLD)

RAG-Based Customer Support Assistant

LangGraph + ChromaDB + Groq LLM + HITL | 2025

1. System Overview

This system is a production-grade RAG (Retrieval-Augmented Generation) Customer Support Assistant. It ingests a PDF/TXT knowledge base, stores embeddings in ChromaDB, and answers user queries using Groq LLaMA via a LangGraph-orchestrated workflow with HITL escalation for sensitive queries.

2. Architecture — Two Pipelines

2.1 Ingestion Pipeline (Offline)

Step	Component	Description
1	Document Loader	Loads PDF/TXT using PyMuPDF or plain text reader
2	Text Chunker	Splits text into 512-char chunks with 50-char overlap
3	Embedding Model	Converts chunks to 384-dim vectors (all-MiniLM-L6-v2)
4	ChromaDB Store	Persists vectors + metadata for semantic retrieval

2.2 Query Pipeline (Online — LangGraph Orchestrated)

Node	Role	Output
intent_node	Detects escalation keywords in query	escalated: True/False
retriever_node	Embeds query, searches ChromaDB top-k	retrieved_docs list
grader_node	Filters top 3 most relevant chunks	relevant_docs list
generator_node	Builds prompt + calls Groq LLaMA	answer + confidence
hitl_node	Escalates sensitive queries to human agent	escalation message
router (edge)	Routes: escalated → HITL, else → retriever	next node decision

3. Component Descriptions

Component	Technology	Purpose
Document Loader	PyMuPDF / plain text	Extract text from PDF/TXT files
Chunker	LangChain RecursiveCharacterTextSplitter	512-char overlapping chunks
Embedding Model	sentence-transformers/all-MiniLM-L6-v2	384-dim local embeddings
Vector Store	ChromaDB (persistent)	Store & search chunk embeddings
LLM	Groq LLaMA 3.1-8b-instant	Generate grounded answers
Workflow Engine	LangGraph StateGraph	Orchestrate nodes + conditional routing
HITL Manager	SQLite-backed HITLManager class	Create & resolve escalation tickets
API/Interface	CLI (rag_app.py)	User interaction & file upload

4. Technology Choices & Scalability

Technology	Why Chosen
------------	------------

ChromaDB	Open-source, local persistence, Python-native, no infra setup needed
LangGraph	Stateful graph workflow, conditional edges, pause/resume for HITL
Groq LLaMA	Ultra-fast inference (~500 tokens/sec), free tier available
sentence-transformers	Free local embeddings, no API cost, 384-dim vectors

- Scalability: Batch ingestion, incremental ChromaDB upsert, Redis caching for common queries
- Latency target: <3s end-to-end | Embedding: ~50ms | Retrieval: ~20ms | LLM: ~1-2s
- Future: Migrate to Pinecone/Weaviate for cloud-scale; deploy via Docker + Kubernetes